

Category B Practice Problems

PROBLEM 1 • Basic

Library Membership Foundation & Batch Enrollment Validator

The college library wants to store its membership records in code. New members are usually added in bulk lists, not one by one. Build `LibraryMember` as the base class with a validated constructor, and `StudentMember` as a simple subclass of it. Then write a method that registers a whole list of member IDs at once and counts how many were rejected, instead of stopping at the first bad one.

Requirements

- `StudentMember` must extend `LibraryMember` directly (single inheritance) and forward the shared fields via `super(...)` — do not duplicate `memberId` or `borrowLimit` as separate fields in `StudentMember`.
- `LibraryMember`'s constructor must reject a `memberId` that's blank, whitespace-only, or shorter than 4 characters.
- `enrollBatch(...)` must attempt to construct one `LibraryMember` per array entry, using `try/catch` to count rejections — it must not pre-validate the strings itself before construction.

Function Signature(s)

```
public LibraryMember(String memberId, int borrowLimit)
public StudentMember(String memberId, int borrowLimit, String course) void
borrowBook()
int getBooksBorrowed()
static String enrollBatch(String[] memberIds, int borrowLimit)
```

Examples

Input	Output
<code>new LibraryMember("LB1", 3)</code>	construction rejected

Explanation: "LB1" is only 3 characters — one short of the minimum.

Input	Output
<code>StudentMember s = new StudentMember("STU10", 3, "CSE");</code> <code>s.borrowBook();</code> <code>s.borrowBook();</code> <code>s.getBooksBorrowed()</code>	2

Input	Output
<code>enrollBatch({"STU1", "LB1", "STU2", " ", "STU3"}, 3)</code>	"Enrolled: 3 Rejected: 2"

HINT Let `LibraryMember`'s constructor be the only place the validation rule lives — `enrollBatch` should simply try to build one, catch the `IllegalArgumentException` if construction fails, and count it as a rejection.

CONSTRAINTS: borrowLimit is a positive integer. memberIds.length up to 300.

PROBLEM 2 • Basic → Intermediate

Three Branches of the Membership Tree

The library is adding two new membership types: an Honors student membership with extra borrowing privileges, and a Faculty membership that is completely separate from students. Build both classes. Then write a method that identifies which type of membership a given object is, and another method that adds up the total books borrowed across a mixed list of memberships.

Requirements

- HonorsStudentMember must extend StudentMember (multilevel inheritance, three classes deep); FacultyMember must extend LibraryMember directly, independent of StudentMember entirely (hierarchical inheritance).
- classifyGeneration(...) must decide its answer using instanceof checks alone — do not add a manual "type" field to any class.
- getTotalBooksBorrowed(...) must accept an array holding any mix of the four membership types and sum their borrowed-book counts using each member's own getBooksBorrowed() — it must not check each member's type before adding it in.

Function Signature(s)

```
public HonorsStudentMember(String memberId, int borrowLimit, String course, int bonusLimit) public
FacultyMember(String memberId, int borrowLimit, String department) static String
classifyGeneration(LibraryMember member)
static int getTotalBooksBorrowed(LibraryMember[] members)
```

Examples

Input	Output
new LibraryMember("STU1", 3).displayInfo() new StudentMember("STU2", 3, "CSE").displayInfo() new HonorsStudentMember("STU3", 3, "ECE", 2).displayInfo() new FacultyMember("STU4", 5, "Physics").displayInfo()	"General Member Books Borrowed: 0" "Student Member Course: CSE Books Borrowed: 0" "Honors Student Member Course: ECE Bonus Limit: 2 Books Borrowed: 0" "Faculty Member Department: Physics Books Borrowed: 0"

Input	Output
classifyGeneration(honorsMember) classifyGeneration(facultyMember)	"Multilevel descendant (3 generations deep)" "Hierarchical sibling (independent branch)"

Input	Output
-------	--------

Explanation: *studentMember* has borrowed 2 books, *honorsMember* has borrowed 1, and *facultyMember* has borrowed 3 — $2 + 1 + 3 = 6$.

HINT `getTotalBooksBorrowed(...)` is a one-line loop body once you trust polymorphism — calling `getBooksBorrowed()` on a `LibraryMember` reference already runs each object's own version, regardless of which subclass it actually is. **CONSTRAINTS:** `members.length` up to 500. Every subclass overrides `displayInfo()` to describe itself.

PROBLEM 3 • Intermediate

The Student Discount & Fine Ledger

Students who return books late are charged a fine, but student members get a discount on every fine. The library also wants a record of every fine ever charged to a membership — one that outside code cannot secretly change.

Requirements

- `StudentMember` must override `chargeFine(int amount)` using `@Override`, calling `super.chargeFine(...)` with the amount halved — it must not reimplement the fine logic from scratch.
- Every fine applied (in the base class or any override) must be recorded into a private array field; `getFineHistory()` must return a defensive copy every time it's called, never the real internal array.

Function Signature(s)

protected void `chargeFine(int amount)` // defined in `LibraryMember`, overridden in `StudentMember`
 int[] `getFineHistory()`
 int `getTotalFine()`

Examples

Input	Output
<pre>StudentMember s = new StudentMember("STU5", 3, "CSE"); s.chargeFine(100); s.getTotalFine()</pre>	50

Explanation: *StudentMember* halves every fine before applying it via *super* — 100 becomes 50.

Input	Output
<pre>int[] history = s.getFineHistory(); history[0] = 999; s.getFineHistory()</pre>	[50]

Explanation: The recorded fine is the halved amount (50), not the original 100 passed in — and tampering with the returned array never reaches the membership's real internal history.

HINT `super.chargeFine(amount / 2)` reuses the parent's own deduction logic and its own array-recording logic in one call — resist writing a second, separate recording step inside `StudentMember`.

CONSTRAINTS: Up to 10 fines per membership. amount is always a positive integer when

`chargeFine` is called. Page 3 of 6

STEP SEM-3 · CodInClub | Powered by BridgeLabz

PROBLEM 4 • Intermediate

The Weekly Circulation Report

Every week, the library prints one report listing all memberships, mixing general and student members together. The report should be built as a single piece of text, not many small print statements. For student memberships specifically, the report should also show their course.

Requirements

- `batchPrint(...)` must loop over a `LibraryMember[]` and call `displayInfo()` polymorphically — no instanceof-based if-else chain deciding what to print.
- The final report must be assembled with `StringBuilder`, not repeated `String` concatenation inside the loop.
- For any membership that is genuinely a `StudentMember`, use instanceof to guard a downcast that reaches its course — attempting the same downcast on a membership that isn't a `StudentMember` must be avoided, not attempted and caught.

Function Signature(s)

`static String batchPrint(LibraryMember[] members)`

Examples

Input	Output
<pre>batchPrint({ new LibraryMember("LB5", 3), new StudentMember("STU6", 3, "ECE") })</pre>	<pre>"General Books: 0 Student Course: ECE Books: 0 [Course via downcast: ECE] "</pre>

Input	Output
<pre>LibraryMember plain = new LibraryMember("LB6", 3); StudentMember bad = (StudentMember) plain;</pre>	<pre>ClassCastException at runtime</pre>

Explanation: This compiles fine — the compiler only checks that *StudentMember* is some kind of *LibraryMember* — but *plain* is genuinely not a *StudentMember* at runtime, so the cast fails the moment it executes.

HINT The instanceof check and the downcast go together as a pair — check first, then cast, and only inside the branch where the check already passed. Append to the same *StringBuilder* across every loop iteration, don't create a new one per membership.

CONSTRAINTS: The array may contain any mix of *LibraryMember* and its subclasses, but never

STEP SEM-3 · CodInClub | Powered by BridgeLabz

PROBLEM 5 • Intermediate → Advanced

Membership Numbers, Renewal Codes & the Nightly Circulation Audit

Problem Statement

The library wants three things handled together: give every membership a unique ID number that can never be changed after it is created; check if a renewal code entered by a member is in the correct format; and go through a batch of memberships at night, safely skipping any entries that are missing.

Requirements

- Every `LibraryMember` must receive a `memberNumber` from a shared static counter, incremented once per object, and stored in a final field — never settable from outside and never reassignable after construction.
- `isValidRenewalCode(...)` must check the exact format "R" + two digits + one uppercase letter (e.g. "R12A") using `charAt()` and `Character.isDigit()/isUpperCase()` — not a regular expression.
- Provide two overloaded `borrowBook(...)` methods: a no-argument version, and a version that also records a genre `String`, reusing the no-argument version internally rather than duplicating its logic.
- `processNightlyAudit(...)` must use `instanceof` to separate `FacultyMember` from a regular `LibraryMember`, and must never throw on a null batch entry.

Function Signature(s)

```
public LibraryMember(int borrowLimit) // assigns a final memberNumber internally void
borrowBook()
void borrowBook(String genre)
static boolean isValidRenewalCode(String code)
static int getMembersEnrolled()
public FacultyMember(int borrowLimit, String department)
static String processNightlyAudit(LibraryMember[] members)
```

Examples

Input	Output
<pre>LibraryMember m1 = new LibraryMember(3); m1.memberNumber LibraryMember.getMembersEnrolled()</pre>	<pre>"LIB-101" 1</pre>

Input	Output
isValidRenewalCode("R12A") isValidRenewalCode("R1A") isValidRenewalCode("X12A")	true false false

Explanation: "R1A" is one character too short; "X12A" has the right shape but the wrong leading letter.

Input	Output
m1.borrowBook(); m1.borrowBook("Fiction"); m1.getBooksBorrowed()	2

Explanation: Both calls land on the same underlying count — the genre overload only records the genre before delegating to the no-argument version.

Input	Output
processNightlyAudit({ new FacultyMember(5, "Physics"), null, new LibraryMember(3) })	"2 processed 1 null skipped 1 faculty 1 regular"

HINT Increment the static counter inside the constructor itself, before building the final memberNumber string from its new value. For the renewal code, check the length first — a wrong length should fail immediately, before you ever call `charAt()` on a position that might not exist.

CONSTRAINTS: Batch size up to 2,000.

